Mizan.ai — Backend Implementation Plan

Auth, JWT & Structured Persistence (v1)

Context document for Claude Code (VS Code) — revision 3

Generated July 17, 2026 · Supersedes revision 2

Every choice in this document is a decision already made, not an option to weigh. Where a decision looks arbitrary, it was made deliberately to match existing code. Do not substitute a different approach without flagging it first. Sections marked **[NEW in v3]** or **[CHANGED in v3]** reverse or add to revision 2 — trust this document over revision 2 where they conflict.

1. Context — What Already Exists

Mizan.ai is an Arabic-first ZATCA/VAT compliance platform for Saudi SMEs. Dual-purpose: a working product AND a portfolio piece for HUMAIN (target employer).

Already built and working — DO NOT MODIFY:

- **modal-serving/** — 4 deployed Modal endpoints (QwenBrain, QwenLite, Translator, Embedder). Inference-only. Never add business logic here.
- **agents/chatbot/** (Feature D) — LangGraph chatbot, confirmed working end-to-end including PostgreSQL persistence. Verified via Swagger: a user profile name persisted and was retrieved correctly across turns using a stable session_id.
- **main.py** — FastAPI gateway. Mock endpoints for Features A, B, C, E, G, H. Feature D (/api/chat) is the only endpoint with real logic wired in.
- **persistence.py** (may live at agents/chatbot/memory.py — see §5.13) — PostgreSQL layer. Two existing schemas: chatbot (table user_activity_log) and work (table customer_memories).

Scope of this task: BACKEND ONLY. No frontend. Login UI, JWT storage in localStorage, and any buttons are a later session.

2. Problem Being Solved

Every chat session currently gets a random UUID session_id. Consequences:

- A returning user cannot be recognised — fresh UUID every visit, all prior context lost.
- There is no persistent user account concept at all.

Decision: add email/password authentication with JWT so returning users are identified by account, not by a throwaway session ID.

3. Core Concepts — Two Different IDs

Keep these strictly separate throughout:

ID	Lifespan	Purpose
user_id	Permanent (tied to account)	Identifies the person across all visits and devices. Lives in auth.users, embedded
session_id	Temporary (one conversation)	Identifies a single conversation thread for LangGraph checkpointing. A fresh UUI

Profile data, filing notes and customer records are keyed by user_id. Conversation checkpoints are keyed by session_id.

4. Decisions Already Made (do not re-litigate)

Topic	Decision	Why
users.id type	TEXT, holding a UUID string generated in Python	Existing user_id columns in chatbot.user_activity_log and work.customer_m
UUID generation	Python-side, never gen_random_uuid()	gen_random_uuid() needs the pgcrypto extension on Postgres < 13. Contai
Schema for users	New auth schema → auth.users	Mirrors the existing schema-per-domain pattern. chatbot.users is semantica
Password hashing	bcrypt directly, NOT passlib	passlib is effectively unmaintained and adds a wrapper layer with no benefit
Password length check	Byte length, NOT character length [CHANGED in v3]	bcrypt truncates at 72 BYTES, not 72 characters. An Arabic password of 40
Email validation	Simple regex, NOT Pydantic EmailStr	EmailStr pulls in email-validator. Standing constraint is minimal dependenci
New dependencies	Exactly two: bcrypt, PyJWT	Nothing else. Do not add passlib, python-jose, email-validator, or an ORM.
Chat memory model	Dual window: 30 stored / 8 sent to model [CHANGED in v3]	Reversed in v2's 'delete [-8:]' instruction. See §5.6 — this is now a deliberate t
Registration behaviour	Register returns a JWT — signup auto-logs-in	Deliberate product decision, not an oversight.
Refresh tokens	Out of scope	24h expiry, re-login when it lapses.
Logout	Out of scope — no backend needed	Stateless JWT: logout is the frontend clearing localStorage.

5. What To Build

5.1 — Auth Schema & Users Table

Add to persistence.py a new function initialize_auth_schema():

```
CREATE SCHEMA IF NOT EXISTS auth;

auth.users
  id TEXT, primary key (UUID string, generated in Python)
  email TEXT, UNIQUE, NOT NULL
  password_hash TEXT, NOT NULL (bcrypt — never plaintext, never logged)
  nickname TEXT
  created_at TIMESTAMP, default CURRENT_TIMESTAMP
  updated_at TIMESTAMP, default CURRENT_TIMESTAMP
```

Where it is called from [CHANGED in v3 — see §5.11 for full ordering]: main.py's lifespan startup hook, and it MUST run before work.user_filing_notes is created (§5.7), because that table has a foreign key to auth.users(id). NOT from ChatbotAgent._init_persistence() — auth is a gateway concern, not a chatbot concern. Leave initialize_memory_schemas() exactly as it is.

Add functions to persistence.py:

- create_user(email, password, nickname) — generate str(uuid.uuid4()) for id, hash password with bcrypt, insert, return user_id
- get_user_by_email(email) — return user record or None
- get_user_by_id(user_id) — return user record or None
- verify_password(plaintext, password_hash) — bcrypt comparison, return bool

bcrypt note [CHANGED in v3]: bcrypt truncates input beyond 72 *bytes*, not 72 characters. Validate using the encoded length, not len(password):

```
if not (8 <= len(password.encode('utf-8')) <= 72):
  raise ValidationError(...)
```

This matters specifically because Mizan.ai is Arabic-first — an Arabic password will run out of the 72-byte budget at roughly half the character count of an English one. A char-length check would let a user register a password that silently truncates, then fail to log in with the exact same password they typed.

5.2 — JWT Logic

New module auth.py (alongside main.py):

- create_jwt(user_id, email, expires_in_hours=24) → signed token string
- verify_jwt(token) → validates signature AND expiry, returns user_id; raises on invalid/expired
- JWT_SECRET loaded from .env — add JWT_SECRET to .env.example

[NEW in v3] Fail loudly if JWT_SECRET is missing. Do not default it to a placeholder string. At import time in auth.py:

```
JWT_SECRET = os.environ["JWT_SECRET"] # KeyError on startup if unset — do not use os.getenv()
with a fallback here
```

[NEW in v3] Docker plumbing. Adding a key to .env.example is not sufficient by itself if docker-compose.yml passes environment variables explicitly (rather than via env_file: .env) — in that case JWT_SECRET must also be added to the service's environment: block or the container will fail this same KeyError at runtime with no indication why. Check docker-compose.yml before assuming .env.example alone is enough.

Lifecycle — where JWT lives:

- Created by: backend, only inside the login and register endpoints
- Sent to: frontend, in the response body
- Stored by: frontend (localStorage) — not our concern this session
- Sent back by: frontend, in the Authorization header of every subsequent request
- Verified by: backend, on every protected request — check signature AND expiry every single time, even for a user who just logged in
- Never stored in Postgres. Postgres holds the account (email + password_hash) only, never tokens.

Every login mints a brand-new JWT, including for a user logging in again the next day. The old token is simply discarded by the frontend. There is no server-side token store and no revocation list in v1.

5.3 — Auth Endpoints (main.py)

POST /api/auth/register

- Input: {email, password, nickname}
- Validate: email regex, email not already registered, password 8–72 **bytes** (see §5.1)
- Hash password, create user, mint JWT
- Output: {user_id, email, jwt_token, message}

POST /api/auth/login

- Input: {email, password}
- Look up by email, verify password hash, mint fresh JWT
- Output: {user_id, email, jwt_token, message}

Error contract — implement exactly this:

Case	Status	Body
Register, email already taken	409	"Email already registered"
Register, malformed email or password outside 8–72 bytes	400	Specific validation message
Login, unknown email OR wrong password	401	"Invalid credentials" — generic, never distinguish the two (account-enumeration l
Any protected route, missing/invalid/expired JWT	401	"Invalid or expired token"

5.4 — JWT Dependency

Add a FastAPI dependency that:

- Reads the Authorization header (Bearer)
- Calls verify_jwt()
- Returns user_id for injection into route handlers
- Raises 401 if missing/invalid/expired

Apply to /api/chat and every future protected endpoint. Leave /health and both auth endpoints unprotected. Wire HTTPBearer into FastAPI's security scheme so the Authorize button appears in Swagger for testing.

5.5 — Refactor /api/chat to Use user_id

Today, session_id doubles as the identity key for store_user_profile() / get_user_profile(). Fix:

The wiring detail that matters: _sync_memory() in langgraph_chatbot.py does not take user_id as a parameter — it reads thread_id out of RunnableConfig:

```
thread_id = (config or {}).get("configurable", {}).get("thread_id")
```

So threading user_id through means passing it via configurable alongside thread_id:

```
config = {"configurable": {"thread_id": session_id, "user_id": user_id}}
```

Changes required:

bullet ChatbotAgent.generate_reply() gains a user_id parameter

bullet config gains the user_id key as above

bullet _sync_memory() reads user_id from config and uses it for store_user_profile() / get_user_profile()

bullet thread_id continues to be used ONLY for LangGraph checkpointing — do not remove it

bullet /api/chat passes user_id (from the JWT dependency) into generate_reply()

5.6 — Chatbot Memory: Dual Window [CHANGED in v3]

This section reverses revision 2. Revision 2 said to delete the [-8:] slice once state was capped at 30, on the reasoning that a single number should be the source of truth. That has been explicitly overridden: keep both numbers, because they solve two different problems.

Layer	Value	What it controls
State (Postgres)	30 messages	How much of the raw transcript is retained in the LangGraph checkpoint before being
Wire (sent to model)	8 messages	How much conversation the model actually sees per request. Directly controls token c

Raising the wire window to match the state cap (i.e. sending all 30 to the model) is a deliberate future change, gated on funding — it roughly quadruples token cost per chatbot call. Do not make this change now. When it happens, it is a one-line change to the slice in _build_wire_messages().

(a) Do NOT remove the [-8:] slice. _build_wire_messages() keeps:

```
for message in state.get("messages", [])[-8:]:
```

(b) Add a trim node to cap Postgres state at 30. add_messages appends — you cannot trim by returning a shorter list. ChatState.messages uses LangGraph's add_messages reducer:

```
messages: Annotated[List[BaseMessage], add_messages]
```

Returning a truncated list from a node will NOT delete anything — the reducer merges by ID and appends. Deletion requires emitting RemoveMessage entries:

```
from langchain_core.messages import RemoveMessage
```

Implement trimming as a dedicated node running after generate_reply, which emits RemoveMessage(id=...) for the oldest messages whenever the list exceeds 30. Do not attempt to trim by list slicing in state.

Net effect: the model only ever has visibility into roughly the last 4 conversational turns (8 messages = ~4 user+assistant pairs), even though up to 30 messages sit in Postgres. Anything the user said earlier than that is invisible to the model unless it was captured into the profile (e.g. their name, via _sync_memory()'s regex extraction, which is independent of both windows).

No permanent persistence of raw chat transcripts beyond 30 messages. No "save session" button in v1.

5.6b — Free-Trial Memory Limit Message [NEW in v3]

Because the model only sees the last 8 messages, a user who references something from earlier in a long conversation will get a response that ignores it entirely — the model has no signal that anything came before what it can see. Rather than let the bot guess or appear confused, tell it to say so explicitly, framed as a free-trial limitation rather than a bug.

Add an instruction to build_system_prompt() in langgraph_chatbot.py: if the user references something that does not appear in the visible message window, the model should use the fallback line below (in the user's language) instead of guessing or fabricating a memory it doesn't have.

English:

“This is a free trial, so I can only remember our recent conversation, not older messages. If you'd like a version with longer memory and more advanced features, reach out to the Mizan.ai team about our Pro plan.”

Arabic:

وليس الرسائل الأقدم. إذا كنت ترغب بذاكرة أطول وميزات متقدمة، تواصل مع فريق . بخصوص خطة .
هذه نسخة تجريبية مجانية، لذلك يمكنني تذكر محادثتنا الأخيرة فقط

Implementation note: this is a system-prompt instruction, not a hardcoded intercept — the model should produce this message (or a close paraphrase in the matching language) when it judges the user is asking about something outside its visible context, not on every single reply.

5.7 — Structured Storage for Advanced Features (Feature E and similar)

Different from the chatbot. Business features (starting with Feature E, the ZATCA compliance calculator) auto-save structured data — no button, no user action. Do NOT store raw conversation transcripts for these features. Store extracted, structured facts only.

Table 1 — work.customer_memories (EXISTS, but current behaviour is WRONG)

store_customer_memory() in persistence.py today performs a plain INSERT with no lookup and no conflict handling. Filing three times for "Ahmed" produces three duplicate rows. updated_at is set once at insert and never touched again.

[CHANGED in v3] Add the unique constraint idempotently. ALTER TABLE ... ADD CONSTRAINT has no IF NOT EXISTS form in Postgres — running it a second time (e.g. on the next container restart, since initialize_memory_schemas() runs on every startup) throws duplicate_object and can break startup. Use a unique index instead, which does support IF NOT EXISTS and which ON CONFLICT accepts:

```
CREATE UNIQUE INDEX IF NOT EXISTS customer_memories_user_customer_uniq
  ON work.customer_memories (user_id, customer_name);
```

The table is currently unused, so adding this now is free. It gets expensive once real duplicate rows exist.

Add upsert_customer_memory(user_id, customer_name, customer_context, issue_summary) using a real ON CONFLICT (user_id, customer_name) DO UPDATE, with these column semantics:

Column	On conflict
customer_context	APPEND — preserve existing, concatenate the new context (running history in one row)
issue_summary	OVERWRITE — latest issue is what matters
updated_at	BUMP to CURRENT_TIMESTAMP
created_at	untouched

[NEW in v3] Guard the append against NULL. On a first-ever insert (no existing row), a naive concatenation of the existing column against the new text returns NULL in Postgres — string concatenation with NULL nulls the whole result, silently discarding the very first entry. Wrap the existing side in COALESCE:

```
customer_context = COALESCE(work.customer_memories.customer_context, '') || E'\n' ||
EXCLUDED.customer_context
```

Keep store_customer_memory() in place, unused. Do not delete it and do not rewrite it in place. upsert_customer_memory() is the new entry point. Removing the old function is a separate cleanup decision, not part of this task.

Table 2 — NEW: work.user_filing_notes

```
work.user_filing_notes
  id SERIAL, primary key
  user_id TEXT, references auth.users(id) (TEXT — matches §4 decision)
  customer_name TEXT
  status TEXT (completed | pending | issues_found)
  notes TEXT (free text — what was done, what's outstanding)
  created_at TIMESTAMP, default CURRENT_TIMESTAMP
```

Note: there is deliberately no separate filing_date column — created_at IS the filing time. Add a distinct filing_date only if a future requirement needs a filing dated differently from when it was entered. One row per filing action, scoped to the user who performed it. This powers "what did I do last time?" queries.

Reminder — see §5.11: this table's FK to auth.users(id) means auth.users MUST exist before this table is created. Do not create it from a code path that could run first.

5.8 — Natural-Language Q&A; Over Filing History

The user can ask, in a text input on a feature page:

- "What did I do last time?"
- "What did I do for customer Mohammed?"
- "What was the VAT amount?" / "What notes did I leave?" (follow-ups)

Build the query layer ONLY. Do not build a graph for this. Feature E has no LangGraph graph yet — the only graph in the codebase is the chatbot's. Inventing a graph here would mean designing Feature E's architecture as a side effect of an auth task. Which graph consumes these functions gets decided when Feature E is actually built.

What to build — plain functions in persistence.py plus a formatting helper:

- No customer named in the question → query work.user_filing_notes filtered by user_id only, most recent row.
- Customer named → query filtered by user_id + customer_name. If no match, respond that this customer has no record — never fabricate an answer.
- Pass the retrieved row(s) plus the user's question to QwenLite to format a natural-language answer.

Testability: nothing writes to work.user_filing_notes yet — Feature E's extraction pipeline does not exist. Ship a small seed script at scripts/seed_filing_notes.py that inserts 3–4 fake rows for a test user, so the query path is verifiable via Swagger before Feature E exists.

Explicitly deferred to v2:

- No Qdrant / embeddings for customer or filing data. Postgres query + LLM formatting is sufficient for v1.
- No "share to memory" button.
- No "get data from memory" button.
- No semantic search over customer history.

The ZATCA/VAT knowledge base (Feature A) is unrelated and continues to use Qdrant as previously planned — unchanged.

5.9 — Leave chatbot.user_activity_log Alone

The table and its helpers (append_user_activity(), get_user_recent_activity()) exist. Save-session is deferred, so nothing writes to it in v1. Instruction: leave the table and both functions exactly as they are. Do not delete them. Do not "helpfully" wire up a writer. They are intentionally dormant.

5.10 — Fix Feature Labelling in main.py (cosmetic, but do it)

main.py's module docstring lists F as voice interaction and G as multi-source synthesis. The synthesis section's comment header says "F — Multi-source document synthesis" while its route tag says G — synthesis. Make it consistent: synthesis is G; F is voice and is cut from v1.

5.11 — Startup Ordering [NEW in v3]

This is a build-breaking issue if skipped, not a style note. main.py currently instantiates ChatbotAgent() at **import time** (module-level, e.g. chatbot_agent = ChatbotAgent()), which runs initialize_memory_schemas() immediately — before the FastAPI lifespan hook ever fires. If initialize_auth_schema() is only called from lifespan (as §5.1 originally specified) and work.user_filing_notes (§5.7) is created anywhere that could run before it, the FK to auth.users(id) will fail because auth.users doesn't exist yet.

Required ordering, enforced explicitly, not by convention:

bullet 1. initialize_auth_schema() — creates auth schema + auth.users

bullet 2. Creation of work.user_filing_notes (whichever function this lives in)

bullet 3. initialize_memory_schemas() (existing chatbot/work schemas — order relative to the above two doesn't matter, since it has no FK dependency on auth.users)

Simplest safe approach: call initialize_auth_schema() from main.py's lifespan hook BEFORE ChatbotAgent() is instantiated, and create work.user_filing_notes inside initialize_auth_schema() itself (or immediately after it, same function scope) rather than as a separate independently-triggered call. If ChatbotAgent() must remain a module-level instantiation for other reasons, move initialize_auth_schema() to run at import time as well, ahead of it — flag this trade-off rather than silently picking one.

5.12 — prompts.py vs. build_system_prompt() [NEW in v3]

The codebase has two separate system-prompt definitions: prompts.py (SYSTEM_PROMPT constant and build_chat_prompt()) and langgraph_chatbot.py's build_system_prompt(). Only build_system_prompt() in langgraph_chatbot.py is actually wired into the live chat path (_generate_reply() → _build_wire_messages()). prompts.py is not imported anywhere in the active flow — it appears to be an earlier draft.

Instruction: build_system_prompt() in langgraph_chatbot.py is authoritative. The §5.6b free-trial fallback message goes there. Do not edit prompts.py to add it — that file is not on the live path and editing it will have no effect. Do not delete prompts.py; leaving an orphaned file alone is out of scope for this task.

5.13 — File Path Note

This document refers to persistence.py throughout. The actual file may be at agents/chatbot/memory.py (confirmed by the import in langgraph_chatbot.py: from agents.chatbot.memory import ...). Search the repo and use whichever path is correct — do not create a second, duplicate persistence file at the literal path persistence.py.

6. Explicit Non-Goals

Do not:

bullet Touch anything in modal-serving/

- Build any frontend UI, login form, or localStorage handling
- Add a "save session" button for the chatbot
- Add "share to memory" or "get data from memory" buttons anywhere
- Add Qdrant/embeddings for customer data
- Build refresh tokens, a logout endpoint, or server-side token revocation
- Build a LangGraph graph for Feature E's Q&A;
- Store raw chat transcripts permanently beyond the 30-message cap for the chatbot
- Delete store_customer_memory(), append_user_activity(), or get_user_recent_activity()
- Implement Features A, B, C, G, H beyond their current mock state
- Add any dependency beyond bcrypt and PyJWT
- Raise the chatbot's wire window above 8 messages (that change is gated on funding — see §5.6)
- Edit prompts.py expecting it to affect the live chatbot (see §5.12)

7. Build Order [UPDATED in v3]

Test each step via Swagger (/docs) before moving on — the same way Feature D's persistence was verified.

1. initialize_auth_schema() + auth.users table + auth helpers, called in the correct order per §5.11
2. work.user_filing_notes table, created after auth.users exists (§5.7, §5.11)
3. auth.py — create_jwt() / verify_jwt(), JWT_SECRET loaded via os.environ[...] (fail loudly if unset), added to .env.example AND docker-compose.yml if needed (§5.2)
4. POST /api/auth/register and POST /api/auth/login in main.py, with the §5.3 error contract and byte-length password validation (§5.1)
5. JWT FastAPI dependency; apply to /api/chat
6. Refactor /api/chat + generate_reply() + _sync_memory() to use user_id via configurable (§5.5)
7. Dual-window chatbot memory: add RemoveMessage trim node capping state at 30, keep [-8:] wire slice unchanged (§5.6)
8. Free-trial memory-limit fallback message, EN + AR, added to build_system_prompt() in langgraph_chatbot.py (§5.6b, §5.12)
9. Unique index (CREATE UNIQUE INDEX IF NOT EXISTS) + upsert_customer_memory() with COALESCE-guarded append on work.customer_memories (§5.7)
10. scripts/seed_filing_notes.py
11. Q&A; query functions + LLM formatting helper (§5.8)
12. Fix F/G labelling in main.py (§5.10)

8. Standing Constraints (carry forward)

- Never put business logic in modal-serving/ — inference-only
- QwenLite for chat/RAG; QwenBrain only for Feature E and tool-calling flows
- The chatbot must never answer ZATCA/VAT questions directly — it redirects to Feature A (unchanged, unrelated to this task)
- Keep dependencies minimal — add per-feature deps only when that feature needs them
- Arabic + English must both work in every user-facing text path
- Passwords always hashed with bcrypt, validated by byte length not character length — never stored, logged, or returned in any response
- Postgres is the system of record for structured data; Qdrant is for the ZATCA knowledge base only in v1
- The chatbot's wire window (8 messages sent to the model) and state window (30 messages retained in Postgres) are two separate, deliberately different numbers — do not collapse them into one